

Random Forest

Modello

Logica dell'Algoritmo

Un **Random Forest** è un **ensemble method** (metodo di insieme) che combina **molteplici decision trees** per ottenere una previsione finale migliore rispetto a un singolo albero.

Idea fondamentale: ridurre l'overfitting e l'instabilità degli alberi singoli tramite:

1. **Bootstrap Aggregating (Bagging):** addestrare ogni albero su un campione bootstrap diverso dei dati
2. **Feature Randomness:** ad ogni split, considerare solo un subset casuale di feature
3. **Averaging/Voting:** combinare le previsioni di tutti gli alberi (media per regressione, majority voting per classificazione)

Procedura di Training

1. **Per ogni albero t da 1 a T (numero di alberi, solitamente 100-1000):**
 - a. Creare un **dataset bootstrap** D_t campionando n istanze **con reinserimento** dai dati originali
 - b. Addestrare un **decision tree** su D_t con i seguenti vincoli:
 - Ad ogni nodo, considerare solo $m_{\text{try}} = \sqrt{p}$ feature (per classificazione) o $m_{\text{try}} = p/3$ (per regressione)
 - Crescere l'albero **completamente** (senza potatura) fino a purezza (overfitting è OK, verrà mitigato dal bagging)
2. **Per la previsione:**
 - **Classificazione:** far votare tutti gli alberi $\rightarrow$ classe finale = moda (più frequente)
 - **Regressione:** far predire tutti gli alberi $\rightarrow$ valore finale = media

Motivazione Teorica

Perché il Bagging funziona?

Se abbiamo un modello con varianza alta (come un albero singolo), combinare molti modelli indipendenti (addestrati su dataset diversi) **riduce la varianza:**

$$\text{Var}(\bar{X}) = \frac{\text{Var}(X)}{N}$$

Se gli alberi fossero completamente indipendenti, la varianza della media calerebbe di un fattore di T (numero di alberi). In pratica non sono indipendenti (correlati perché addestrati sugli stessi dati), quindi la riduzione è minore, ma comunque significativa.

Perché la Randomness nelle Feature?

Il termine m_{try} introduce **de-correlazione** tra gli alberi:

- Se usassero tutte le feature, tutti gli alberi farebbero gli stessi split iniziali (molto correlati)
- Considerando solo $\sqrt{p}$ feature casuali, alberi diversi imparano dipendenze diverse $\rightarrow$ sono meno correlati
- Alberi meno correlati $\rightarrow$ riduzione della varianza nel voting

Complessità Computazionale

Training

$$O(T \cdot n \log(n) \cdot p)$$

Dove:

- T = numero di alberi (solitamente 100-1000, quindi fattore significativo)
- n = numero di osservazioni
- $\log(n)$ = profondità media di un albero bilanciato (senza potatura)
- p = numero di feature (ma con feature randomness, solo $\sqrt{p}$ considerate)

Spiegazione:

- Per ogni albero (T iterazioni):
 - Creare bootstrap: $O(n)$
 - Trainare albero: $O(n \log(n) \cdot m_{\text{try}})$ dove $m_{\text{try}} = \sqrt{p} \rightarrow O(n \log(n) \sqrt{p})$
- **Total:** $O(T \cdot n \log(n) \sqrt{p})$

Parallelizzazione

Random Forest è **intrinsecamente parallelizzabile**:

- Ogni albero è indipendente $\rightarrow$ addestrare su processori diversi
- Con K processori, speedup $\approx K$ (idealmente)
- Pratica standard: usare tutti i core disponibili

Inference (Previsione)

$$O(T \cdot d)$$

Dove d è la **profondità media di un albero**.

Per alberi non potati, $d \approx \log(n)$, quindi:

$$O(T \log(n))$$

Confronto con singolo albero: $O(\log(n))$ vs $O(T \log(n))$ — inference è più lenta di un albero singolo, ma comunemente accettabile.

Memoria

$$O(T \cdot n \cdot d)$$

Memorizzare T alberi, ognuno con $O(n)$ nodi nel peggiore (completo):

Per alberi bilanciati: $O(T \cdot n)$ (lineare in T).

Scalabilità: Conclusioni

Vantaggi:

- Training scala bene con n (logaritmico) e p (sqrt)
- Parallelizzazione è naturale e efficiente
- Può gestire dataset molto grandi (milioni di istanze)

Svantaggi:

- fattore T (numero di alberi) aggiunge overhead
- Memoria cresce linearmente con T
- Per interpretabilità, T grande rende difficile ispezionare

Pratica: Random Forest è considerato uno dei migliori compromessi tra performance e scalabilità, secondo solo ai metodi gradient-based (Gradient Boosting, LightGBM) per dataset molto grandi.

Rappresentazione Interna

Struttura del Modello

Un Random Forest è rappresentato come:

Forest = [Tree_1, Tree_2, ..., Tree_T]

Dove ogni Tree_i è un Decision Tree completo (no pruning)

Internamente:

- **T alberi indipendenti**, ognuno con una struttura ricorsiva (nodo → split → figli)
- **Ogni albero ha:** feature indice, threshold, puntatori ai figli, predizioni nelle foglie

Implicazioni per la Spiegabilità

Contro:

- **Molto opaco globalmente:** con T = 100-500 alberi, è impossibile ispezionare manualmente il modello completo
- **Difficile tracciare ragionamento:** non puoi seguire una singola catena di decisioni (ci sono 100 catene parallele)
- **Aggregazione nascosta:** il voting/averaging che determina la previsione finale non è facilmente interpretabile

Pro:

- **Interpretabile localmente:** puoi estrarre il singolo albero più importante (quello che spiega meglio una previsione)
- **Feature importance naturale:** misurata dalla riduzione di impurità media su tutti gli alberi
- **Out-of-Bag (OOB) error:** stima di generalizzazione gratuita, senza bisogno di validation set separato

Confronto con altri modelli:

- **DT singolo:** interpretabile globalmente, ma instabile
- **Random Forest:** opaco globalmente, ma stabile; interpretabilità per feature
- **SVM:** completamente opaco, nessuna feature importance naturale

Vincoli sui Dati

Nessuna Assunzione Strutturale

Come Decision Trees singoli, Random Forest **non ha assunzioni** su:

- Linearità
- Normalità
- Omoschedasticità
- Indipendenza (relativa)

Bilanciamento delle Classi

Random Forest eredita il problema dei Decision Trees per **classi sbilanciate**:

- Il voting del majority può favorire la classe maggioritaria
- Su dataset 95% negativi / 5% positivi, il modello potrebbe predire sempre negativo

Soluzioni:

- **Class weights:** pesi inversi alla frequenza per ogni albero
- **Stratified bootstrap:** assicurare che ogni bootstrap mantenga la proporzione di classi
- **Threshold tuning:** non usare 50% come confine, usare una soglia ottimale

Feature di Tipi Diversi

Random Forest gestisce **feature categoriche e numeriche** naturalmente:

- Non richiede encoding (come LR, LogR)
- Split su feature categoriche sono gestiti automaticamente
- Nessun vincolo di scala (a differenza di SVM)

Nessun Vincolo sulla Multicollinearità

A differenza di LR/LogR, Random Forest **non è sensibile a multicollinearità**:

- Se due feature sono correlate, l'albero semplicemente ne usa una
- Non causa instabilità come nei modelli lineari

Capacità Predittive

Punti di Forza

1. Eccellente Performance su Dati Reali

- Spesso outperform molti altri algoritmi su dataset variegati
- Cattura pattern non lineari e interazioni naturalmente

2. Robusto a Outlier e Rumore

- Gli outlier influenzano alcuni alberi, non tutti
- Voting/averaging mitiga l'impatto

3. Gestisce Pattern Complessi

- Interazioni tra feature
- Feature categoriche e numeriche miste
- Relazioni non monotone

4. Riduce Overfitting dei Singoli Alberi

- Bagging + feature randomness mitiga fortemente l'instabilità
- Più stabile di un singolo albero

5. Feature Importance Nativa

- Misura diretta dell'importanza di ogni feature
- Utile per feature selection e interpretazione

6. Scalabilità

- Parallelizzabile
- Gestisce dataset grandi

Punti di Debolezza

1. Interpretabilità Scarsa Globalmente

- Con 100-500 alberi, è impossibile ispezionare il modello
- Black box dal punto di vista dell'interpretazione globale

2. Sensibilità a Feature Dominanti

- Se una feature ha molte categorie, l'albero può over-splitarvi
- Bias verso feature ad alta cardinality (come DT singolo)

3. Memoria e Tempo di Inference

- Più lento di un singolo albero o di modelli lineari
- Memoria proporzionale a $T \times$ dimensioni albero

4. Hyperparameter Tuning

- Parametri principali: T (numero alberi), max_depth , min_samples_split , m_try
- Tuning male = performance ridotta

5. Predizione Non Probabilistica per Classificazione

- Output è probabilità (frazione di alberi che votano classe) — non calibrata come LogR

Metriche per la Confidenza

Metriche di Classificazione Standard

Stesse metriche di altri classificatori:

- **Confusion Matrix:** TP, TN, FP, FN
- **Accuracy:** $(TP + TN) / (TP + TN + FP + FN)$
- **Precision:** $TP / (TP + FP)$
- **Recall/Sensitivity:** $TP / (TP + FN)$
- **Specificity:** $TN / (TN + FP)$
- **F1-Score:** media armonica di Precision e Recall
- **ROC Curve e AUC:** trade-off tra TPR e FPR

Probabilità di Classe (Voting)

Random Forest produce naturalmente **probabilità della classe** per classificazione:

$$P(y = k) = \frac{\text{numero di alberi che votano classe } k}{T}$$

Interpretazione:

- Se 80 su 100 alberi votano "Positivo", $P(y = 1) = 0.80$
- Questa è una **stima di confidenza** della previsione

Caveat: queste probabilità non sono ben calibrate (non riflettono accuratezza vera come in LogR).

Per probabilità calibrate, usare Platt scaling o isotonic regression post-hoc.

Out-of-Bag (OOB) Error

Misura di validazione **gratuita** senza validation set separato:

Per ogni istanza di training, gli alberi addestrati su bootstrap che **non contengono** quell'istanza (out-of-bag) la predicono. OOB error = errore medio su queste predizioni.

$$\text{OOB Error} = \frac{1}{n} \sum_{i=1}^n \mathbb{1}[\text{predizione OOB}_i \neq y_i]$$

Vantaggi:

- Stima di generalizzazione senza bisogno di validation set
- Correlato con test error (anche se non identico)
- Utile per tuning di hyperparameter

Metriche di Regressione

Per regressione:

- **Mean Squared Error (MSE):** $\frac{1}{n} \sum (y_i - \hat{y}_i)^2$
- **Root Mean Squared Error (RMSE):** $\sqrt{MSE}$
- **Mean Absolute Error (MAE):** $\frac{1}{n} \sum |y_i - \hat{y}_i|$
- **R²:** frazione di varianza spiegata
- **OOB RMSE:** alternativa a cross-validation

Metriche per la Comprensione e Spiegabilità

1. Feature Importance (Impurità Media)

La metrica più importante di Random Forest per l'interpretazione:

$$\text{Importance}_j = \frac{1}{T} \sum_{t=1}^T \sum_{\text{nodi dove feature } j \text{ fa split}} \text{IG}_{t, \text{nodo}}$$

Dove IG è l'**information gain** (riduzione di impurità).

Interpretazione:

- Feature con importanza alta contribuiscono molto alle decisioni
- Ordinare per importanza identifica le feature più rilevanti
- Utile per feature selection e comunicazione ai non-esperti

Visualizzazione: bar plot di feature ordinate per importanza.

2. Feature Importance (Permutation-Based)

Alternativa al metodo di impurità:

- Permutare casualmente una feature nei dati di test
- Misurare il degrado in performance
- Grandi degni = feature importante

Vantaggio: meno biased verso feature ad alta cardinality (come il metodo di impurità).

3. Partial Dependence Plot (PDP)

Mostra come la previsione media varia al variare di una feature:

$$\text{PDP}_j(x_j) = \frac{1}{n} \sum_{i=1}^n \hat{f}(x_j, x_{-j}^{(i)})$$

Dove $x_{-j}^{(i)}$ sono i valori di altre feature dall'istanza i, mantenendo x_j fisso.

Visualizzazione: linea o curva che mostra l'effetto della feature sulla previsione.

Caveat: se la feature è correlata con altre, l'effetto può essere spurio.

4. Individual Conditional Expectation (ICE) Plot

Versione per-istanza di PDP:

- Mostrare come la previsione cambia al variare di una feature per singoli campioni
- Linee per campione (invece che media su tutti)

Utilità: capire se l'effetto di una feature è uniforme o varia tra istanze.

5. SHAP (SHapley Additive exPlanations)

Metodo avanzato per spiegabilità:

- Assegnare ogni output tra le feature usando valori Shapley dalla teoria dei giochi
- Applicabile a qualsiasi modello
- Produce spiegazioni teoricamente fondate

Vantaggio: considerare interazioni e correlazioni tra feature.

6. Single Tree Extraction

Per una previsione specifica:

- Estrarre il singolo albero che ha la maggiore "responsabilità" della previsione
- Mostrare il cammino dalla radice alla foglia di quell'albero
- Comunicare il ragionamento tramite quello specifico albero

Limite: può essere fuorviante perché ignora il contributo degli altri alberi.

Limiti di Predizione

Sensibilità a Feature Dominanti

Come Decision Trees, Random Forest tende a preferire feature con **molte categorie o valori unici**:

- Feature categoriche con 100 valori vs feature numerica continua
- Questo bias persiste anche con feature randomness

Mitigazione: usare permutation-based feature importance, che è meno biased.

Difficoltà con Dati Molto Sbilanciati

Se le classi sono estremamente sbilanciate (99% vs 1%):

- Il majority voting favorisce la classe maggioritaria
- Accuratezza può essere alta ma recall sulla classe rara bassa

Mitigazione: usare class weights, stratified bootstrap, o threshold tuning.

Scarsa Extrapolazione

Random Forest è un **interpolatore locale**:

- Non può predire valori significativamente fuori dal range di training
- Per regressione, le predizioni rimangono nel range dei dati di training
- Non adatto per trend/trend extrapolation

Performance su Feature Numeriche Pure

Su dataset con solo feature numeriche e pattern lineare, modelli lineari (LR) superano spesso Random Forest:

- RF "impara" la linearità tramite molti split ortogonali → inefficiente
- LR cattura la linearità con pochi parametri

Sensibilità a Iperparametri

Sebbene meno sensibile di SVM, Random Forest ha iperparametri critici:

- **T (numero alberi):** troppo pochi = underfitting; non ha limite superiore strict
- **max_depth:** alberi troppo profondi = overfitting; troppo poco profondi = underfitting
- **min_samples_split:** controlla quando smettere di splittare
- **m_try (feature per split):** influenza la diversità tra alberi

Limiti di Spiegabilità

Opacità Globale

Con $T = 100-500$ alberi:

- **Impossibile ispezionare manualmente** il modello completo
- Non puoi tracciare una singola sequenza di decisioni (ce ne sono T parallele)
- Il modello rimane una "**scatola nera**" per la maggior parte degli utenti

Contrasto con DT singolo: un albero piccolo è completamente trasparente; un RF non lo è.

Interpretazione di Feature Importance Ambigua

Feature importance (impurità) conta quante volte una feature fa split, non necessariamente quanto sia causale:

- Una feature correlata ad un'altra può avere bassa importanza se l'altra è splittata per prima
- Importanza non implica causalità

Soluzione: usare permutation-based importance (meno biased) o SHAP.

Interazioni Non Esplicite

Random Forest cattura interazioni automaticamente (feature A influenza come feature B splitta), ma:

- Le interazioni rimangono implicite nella struttura dell'albero
- Difficile comunicare "feature A e B interagiscono così"

Soluzione: usare SHAP per analizzare interazioni specifiche.

Dipendenza da Iperparametri

La spiegazione fornita da feature importance dipende da:

- `max_depth` (alberi più profondi = più split su feature rare)
- `m_try` (feature randomness influenza quali feature sono considerate)
- T (più alberi = più stabile, meno varianza)

Se si cambia iperparametri, feature importance cambia → spiegazione non è robusta.

Difficoltà di Spiegazione per Non-Esperti

Comunicare il risultato di RF a un non-esperto richiede:

- Tradurre "feature importance" in linguaggio intuitivo
- Evitare di mostrare 100 alberi
- Usare visualizzazioni (SHAP, PDP) per semplificare

Confronto con altri Algoritmi

Vs. Decision Tree Singolo

Aspetto	DT Singolo	Random Forest
Interpretabilità	★★★★★ Altissima	★★ Bassa
Overfitting	★★★★★ (alto)	★ (basso, mitigato)
Stabilità	★ (instabile)	★★★★★ (stabile)
Performance	★★★★ Media	★★★★★ Eccellente
Scalabilità	★★★★ Buona	★★★★★ Buona (parallelizzabile)
Quando usare	Max interpretabilità	Quando performance conta

Vs. Logistic Regression

Aspetto	LogR	Random Forest
Interpretabilità	★★★★★ Alta	★★ Bassa
Performance Dati Lineari	★★★★★ Eccellente	★★ Inefficiente
Performance Dati Non-Lineari	★ Scarsa	★★★★★ Eccellente
Feature Categoricalhe	★ (richiede encoding)	★★★★★ Naturale
Quando usare	Pattern lineare + interpretabilità	Pattern complesso, features miste

Vs. SVM

Aspetto	SVM	Random Forest
Scalabilità	★★ Scarsa $O(n^3)$	★★★★★ Buona
Performance	★★★★★ Buona	★★★★★ Spesso migliore
Interpretabilità	★★ Scarsa	★★ Scarsa
Hyperparameter Tuning	★★ Sensibile	★★★★ Meno sensibile
Feature Importance	✗ No naturale	✓ Sì, nativo
Quando usare	Dati piccoli/medi stabili	Dati grandi, performance critica

Vs. Gradient Boosting

Aspetto	Gradient Boosting	Random Forest
Performance	★★★★★ Spesso migliore	★★★★★ Buona
Velocità Training	★★ Lenta (sequenziale)	★★★★★ Veloce (parallela)
Hyperparameter Tuning	★ Difficile (molti parametri)	★★★★ Più semplice
Risk Overfitting	★★ Alto	★ Basso
Quando usare	Massima performance (con tuning)	Performance buona, implementazione semplice

Vs. Neural Networks

Aspetto	Neural Networks	Random Forest
Capacità	★★★★★ Illimitata	★★★★★ Molto buona
Dati Necessari	★ Molto (milioni)	★★★★★ Meno (migliaia)
Feature Importance	✗ Molto difficile	✓ Nativa
Interpretabilità	★ Nessuna	★★ Poca
Training	★★ Lungo, GPU-friendly	★★★★ Veloce, CPU
Quando usare	Big data, no interpretabilità	Medium data, need feature importance

Varianti e Estensioni

1. Extra Trees (Extremely Randomized Trees)

Simile a Random Forest, ma con maggiore randomness:

- Split threshold è **casuale** (non cerca il miglior threshold)
- Meno computazionalmente costoso (no search del threshold)
- Spesso produce alberi meno profondi e più generali
- Trade-off: performance leggermente minore, ma velocità molto maggiore

2. Gradient Boosting (XGBoost, LightGBM, CatBoost)

Evoluzione sequenziale di Random Forest:

- Anziché addestrare alberi in parallelo (indipendenti), li addestra in **sequenza**
- Ogni albero corregge gli errori del precedente
- Spesso outperform RF, ma richiede tuning più accurato
- Più lento per training ma spesso migliore in performance

3. Isolation Forest

Specializzato per **anomaly detection**:

- Alberi costruiti per isolare anomalie (non per classificazione regolare)
- Anomalie sono isolate in meno split rispetto a punti normali
- Veloce e efficiente per alta dimensionalità

4. Random Forest per Regression

Applica la stessa logica ma:

- Predizione finale è **media** (non voting)
- Feature importance basata su riduzione MSE (non impurità)
- Output naturalmente continuo

Raccomandazioni Pratiche

Quando Usare Random Forest

✓ Ottime scelte:

- Dataset di qualsiasi taglia (piccolo a medio)
- Pattern non lineari e complessi
- Necessità di feature importance
- Features miste (categoriche e numeriche)
- Quando interpretabilità non è critica ma performance sì

✗ Cattive scelte:

- Massima interpretabilità richiesta (es. medicina regolata, finanza)
- Dati puramente lineari (LR è più efficiente)
- Inferenza in real-time su devices con risorse limitate (T alberi è lento)
- Quando è critico capire il "perché" di ogni singola previsione

Hyperparameter Tuning Consigliato

1. Numero di alberi (**n_estimators**):

- Iniziare con 100-200
- Aumentare fino a quando OOB error non migliora
- Diminishing returns dopo 500-1000

2. Profondità massima (**max_depth**):

- Iniziare senza limite (None), poi ridurre se overfitting
- Tipicamente: 10-20 per dataset di taglia media

3. Min samples per split (**min_samples_split**):

- Default: 2 (troppo aggressivo)
- Provare: 5, 10, 20
- Aumentare per ridurre overfitting

4. Feature per split (**max_features**):

- Classification: $\sqrt{p}$ (default, di solito ottimale)
- Regression: $p/3$ (default)
- Provare: 'auto', 'sqrt', 'log2'

5. Class weights (se classi sbilanciate):

- 'balanced' per pesi automatici (inversi alla frequenza)

Miglior Pratica di Validazione

1. Dividere in train (70-80%), test (20-30%)
2. Training su train set con OOB error per validazione interna
3. Validare su test set separato
4. Cross-validation (5-10 fold) per più stabilità

Feature Engineering

Random Forest è **resistente a feature engineering scadente**:

- Non richiede scaling
- Gestisce feature categoriche automaticamente
- Non sensibile a feature correlate

Quindi focus su:

- Feature relevance (sono importanti per il problema?)
- Feature nuove (es. interazioni derivate manualmente) se performance non è sufficiente

Prompt